



Documentation

signotec WebSocket Pad Server

Application for communicating with signotec
Sigma, Omega, Gamma, Delta and Alpha

Version: 2.10

Date: 07.06.2018

© signotec GmbH

www.signotec.de

Tel.: (+49-2102) 535-7510

E-mail: info@signotec.de

Contents

1	DOCUMENT HISTORY	5
2	FUNCTION OVERVIEW	5
3	SYSTEM REQUIREMENTS FOR WINDOWS	7
4	SYSTEM REQUIREMENTS FOR LINUX	8
5	INSTALLATION OPTIONS	9
5.1	OPTION 1 (LOCAL.SIGNOTECWEBSOCKET.DE)	9
5.2	OPTION 2 (LOCAL.SIGNOTECWEBSOCKET.DE, LOCALHOST AND 127.0.0.1)	9
5.3	OPTION 3 (LOCALHOST AND 127.0.0.1)	10
5.4	ENABLE EDGE SUPPORT (ALLOW LOOPBACK)	10
5.5	REMOVE THE OLD CA CERTIFICATE FROM THE TRUSTED ROOT CERTIFICATION AUTHORITIES LIST	10
5.6	CONFIGURE THE CONNECTION-URL	11
5.7	SILENT INSTALLATION	11
5.8	INSTALLATION UNDER LINUX	12
6	LOCAL CONNECTION TYPE OF PADS	14
6.1	USER-SPECIFIC CONFIGURATION FILE FOR CONNECTION TYPE	14
6.2	SYSTEM-WIDE CONFIGURATION FILE FOR CONNECTION TYPE	15
6.3	INSTALLATION OF PADCONFIG.INI	15
7	STANDARD AND API MODE	16
7.1	STANDARD MODE	16
7.2	API MODE	16
7.2.1	<i>Numbers</i>	16
7.2.2	<i>Enums</i>	16
7.2.3	<i>Boolean parameters</i>	17
7.2.4	<i>Colours</i>	17
7.2.5	<i>Pointer parameters</i>	17
7.2.6	<i>Byte arrays and images</i>	18
7.2.7	<i>Files</i>	18
8	COMMUNICATION SETUP	19
8.1	TYPES (TOKEN_TYPE KEY)	19
8.1.1	<i>Requests (TOKEN_TYPE_REQUEST value)</i>	19
8.1.2	<i>Responses (TOKEN_TYPE_RESPONSE value)</i>	20
8.1.3	<i>Events (TOKEN_TYPE_SEND value)</i>	21
8.2	COMMANDS (TOKEN_CMD AND TOKEN_CMD_ORIGIN KEYS)	21
8.2.1	<i>TOKEN_CMD_SET_INTERFACE_VERSION value</i>	21
8.2.2	<i>TOKEN_CMD_GET_SERVER_VERSION value</i>	22
8.2.3	<i>TOKEN_CMD_SEARCH_FOR_PADS value</i>	22
8.2.4	<i>TOKEN_CMD_OPEN_PAD value</i>	23
8.2.5	<i>TOKEN_CMD_SELECTION_DIALOG value</i>	23
8.2.6	<i>TOKEN_CMD_SELECTION_CHANGE value</i>	24
8.2.7	<i>TOKEN_CMD_SELECTION_CONFIRM value</i>	24
8.2.8	<i>TOKEN_CMD_SELECTION_CANCEL value</i>	25
8.2.9	<i>TOKEN_CMD_SIGNATURE_START value</i>	25
8.2.10	<i>TOKEN_CMD_SIGNATURE_RETRY value</i>	25
8.2.11	<i>TOKEN_CMD_SIGNATURE_STOP value</i>	26
8.2.12	<i>TOKEN_CMD_SIGNATURE_CONFIRM value</i>	26
8.2.13	<i>TOKEN_CMD_SIGNATURE_CANCEL value</i>	27
8.2.14	<i>TOKEN_CMD_SIGNATURE_SIGN_DATA value</i>	27
8.2.15	<i>TOKEN_CMD_SIGNATURE_IMAGE value</i>	28

8.2.16	TOKEN_CMD_CLOSE_PAD value	28
8.2.17	TOKEN_CMD_SIGNATURE_POINT value	29
8.2.18	TOKEN_CMD_DISCONNECT value	29
8.2.19	TOKEN_CMD_ERROR value	29
8.2.20	TOKEN_CMD_API_DEVICE_SET_COM_PORT value	30
8.2.21	TOKEN_CMD_API_DEVICE_GET_CONNECTION_TYPE value	30
8.2.22	TOKEN_CMD_API_DEVICE_GET_COM_PORT value	30
8.2.23	TOKEN_CMD_API_DEVICE_GET_IP_ADDRESS value	30
8.2.24	TOKEN_CMD_API_DEVICE_GET_COUNT value	31
8.2.25	TOKEN_CMD_API_DEVICE_GET_INFO value	31
8.2.26	TOKEN_CMD_API_DEVICE_GET_VERSION value	31
8.2.27	TOKEN_CMD_API_DEVICE_OPEN value	32
8.2.28	TOKEN_CMD_API_DEVICE_CLOSE value	32
8.2.29	TOKEN_CMD_API_SENSOR_GET_SAMPLE_RATE_MODE value	32
8.2.30	TOKEN_CMD_API_SENSOR_SET_SAMPLE_RATE_MODE value	32
8.2.31	TOKEN_CMD_API_SENSOR_SET_SIGN_RECT value	33
8.2.32	TOKEN_CMD_API_SENSOR_CLEAR_SIGN_RECT value	33
8.2.33	TOKEN_CMD_API_SENSOR_ADD_HOT_SPOT value	33
8.2.34	TOKEN_CMD_API_SENSOR_SET_HOT_SPOT_MODE value	34
8.2.35	TOKEN_CMD_API_SENSOR_CLEAR_HOT_SPOTS value	34
8.2.36	TOKEN_CMD_API_SENSOR_HOT_SPOT_PRESSED value	34
8.2.37	TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION value	34
8.2.38	TOKEN_CMD_API_SIGNATURE_START value	35
8.2.39	TOKEN_CMD_API_SIGNATURE_STOP value	35
8.2.40	TOKEN_CMD_API_SIGNATURE_CONFIRM value	35
8.2.41	TOKEN_CMD_API_SIGNATURE_RETRY value	36
8.2.42	TOKEN_CMD_API_SIGNATURE_CANCEL value	36
8.2.43	TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA value	36
8.2.44	TOKEN_CMD_API_SIGNATURE_SAVE_AS_STREAM_EX value	36
8.2.45	TOKEN_CMD_API_SIGNATURE_GET_BOUNDS value	37
8.2.46	TOKEN_CMD_API_SIGNATURE_GET_STATE value	37
8.2.47	TOKEN_CMD_API_DISPLAY_ERASE value	37
8.2.48	TOKEN_CMD_API_DISPLAY_ERASE_RECT value	38
8.2.49	TOKEN_CMD_API_DISPLAY_CONFIG_PEN value	38
8.2.50	TOKEN_CMD_API_DISPLAY_SET_FONT value	38
8.2.51	TOKEN_CMD_API_DISPLAY_SET_FONT_COLOR value	39
8.2.52	TOKEN_CMD_API_DISPLAY_SET_TARGET value	39
8.2.53	TOKEN_CMD_API_DISPLAY_SET_TEXT value	39
8.2.54	TOKEN_CMD_API_DISPLAY_SET_TEXT_IN_RECT value	40
8.2.55	TOKEN_CMD_API_DISPLAY_SET_IMAGE value	40
8.2.56	TOKEN_CMD_API_DISPLAY_SET_IMAGE_FROM_STORE value	40
8.2.57	TOKEN_CMD_API_DISPLAY_SAVE_IMAGE_AS_STREAM value	41
8.2.58	TOKEN_CMD_API_DISPLAY_SET_STANDBY_IMAGE value	41
8.2.59	TOKEN_CMD_API_DISPLAY_CONFIG_SLIDE_SHOW_EX value	41
8.2.60	TOKEN_CMD_API_DISPLAY_GET_STANDBY_ID value	42
8.2.61	TOKEN_CMD_API_DISPLAY_GET_WIDTH value	42
8.2.62	TOKEN_CMD_API_DISPLAY_GET_HEIGHT value	42
8.2.63	TOKEN_CMD_API_DISPLAY_SET_ROTATION value	42
8.2.64	TOKEN_CMD_API_DISPLAY_GET_ROTATION value	43
8.3	PARAMETER	43
8.3.1	TOKEN_PARAM_RETURN_CODE key	43
8.3.2	TOKEN_PARAM_ERROR_DESCRIPTION key	44
8.3.3	TOKEN_PARAM_ERROR_CONTEXT key	44
8.3.4	TOKEN_PARAM_PAD_SUBSET key	44
8.3.5	TOKEN_PARAM_CONNECTED_PADS key	45

8.3.6	TOKEN_PARAM_PAD_INDEX key	46
8.3.7	TOKEN_PARAM_PAD_TYPE key	46
8.3.8	TOKEN_PARAM_PAD_CONNECTION_TYPE key	46
8.3.9	TOKEN_PARAM_PAD_COM_PORT key	46
8.3.10	TOKEN_PARAM_PAD_IP_ADDRESS key	47
8.3.11	TOKEN_PARAM_PAD_SERIAL_NUMBER key	47
8.3.12	TOKEN_PARAM_PAD_FIRMWARE_VERSION key	47
8.3.13	TOKEN_PARAM_FIELD_NAME key	47
8.3.14	TOKEN_PARAM_CUSTOM_TEXT key	47
8.3.15	TOKEN_PARAM_SIGNATURE_SIGN_DATA key	48
8.3.16	TOKEN_PARAM_FILE_TYPE key	48
8.3.17	TOKEN_PARAM_PEN_WIDTH key	48
8.3.18	TOKEN_PARAM_FILE key	49
8.3.19	TOKEN_PARAM_POINT key	49
8.3.20	TOKEN_PARAM_VERSION key	49
8.3.21	TOKEN_PARAM_PAD_X_RESOLUTION key	50
8.3.22	TOKEN_PARAM_PAD_Y_RESOLUTION key	50
8.3.23	TOKEN_PARAM_PAD_DISPLAY_WIDTH key	50
8.3.24	TOKEN_PARAM_PAD_DISPLAY_HEIGHT key	51
8.3.25	TOKEN_PARAM_PAD_SAMPLING_RATE key	51
8.3.26	TOKEN_PARAM_PAD_CAPABILITIES key	51
8.3.27	TOKEN_PARAM_FIELD_LIST key	51
8.3.28	TOKEN_PARAM_FIELD_ID key	52
8.3.29	TOKEN_PARAM_FIELD_TEXT key	52
8.3.30	TOKEN_PARAM_FIELD_CHECKED key	52
8.3.31	TOKEN_PARAM_FIELD_REQUIRED key	52
8.3.32	TOKEN_PARAM_PAD_ENCRYPTION key	53
8.3.33	TOKEN_PARAM_DOCHASH key	53
8.3.34	TOKEN_PARAM_ENCRYPTION_CERT key	53
8.3.35	TOKEN_PARAM_CERT_ID key	53
8.3.36	TOKEN_PARAM_ROTATION key	53
8.3.37	Other keys	54

Legal notice

All rights reserved. This document and the components it describes are products copyrighted by signotec GmbH, based in Ratingen, Germany. Reproduction of this documentation, in part or in whole, is subject to prior written approval from signotec GmbH. All hardware and software names used are trade names and/or trademarks of their respective manufacturers/owners. Subject to change at any time without notice. We assume no liability for any errors that may appear in this documentation.

1 Document history

Version	Date	Person responsible	Status/note
1.11	22 April 2015	Paul Grütter	Document structure revised
1.12	29 April 2015	Valentin Tkachev	Information relating to Mozilla Firefox TLS adjusted
1.13	21 October 2015	Andreas Brockmüller	'Information on use in Citrix environments' and 'Silent installation parameters' added.
1.14	26 October 2015	Valentin Tkachev	'Silent installation parameters' added.
1.15	03 December 2015	Valentin Tkachev	Sections 4 and 5.6 added. Sections 6.2.1, 6.2.9, 6.3.8, 6.3.9 changed.
1.16	17 March 2017	Paul Grütter	Sections 6.3.3 and 6.3.6 changed.
2.0	28 March 2017	Paul Grütter	Sections 6, 7.2.15–7.2.41, 7.3.24 added. Sections 3, 5.2, 7, 7.1.1, 7.1.3, 7.2.1, 7.3.1, 7.3.12, 7.3.13, 7.3.17, 7.3.19, 7.3.20, 7.3.21, 7.3.22, 7.3.23 changed.
2.1	5 April 2017	Paul Grütter	Sections 7.2.16–7.2.18, 7.2.24, 7.2.25, 7.2.29, 7.2.40–7.2.46, 7.2.52–7.2.55, 7.3.16 added. Section 7.2.33 removed. Sections 7.1.1 and 7.2.11 changed.
2.2	19 April 2017	Paul Grütter	Section 7.3.6 changed.
2.3	9 May 2017	Valentin Tkachev	Sections 3, 5.2, 5.3 changed.
2.4	30 January 2018	Valentin Tkachev	Sections 3, 4, 5, 5.1, 5.4 changed.
2.5	28 February 2018	Valentin Tkachev	Sections 5.4, 6, 8.2.58, 8.2.59, 8.3.25, 8.3.26, 8.3.27, 8.3.28, 8.3.29, 8.3.30 added. Sections 5.2, 5.5, 8.2.5, 8.3.4, 8.3.14 changed.
2.6	27 March 2018	Valentin Tkachev	Sections 5.5 added.
2.7	20 April 2018	Valentin Tkachev	Sections 5.6 added. Sections 5.1, 5.7 changed.
2.8	04 May 2018	Paul Grütter	Sections 8.2.5, 8.2.6, 8.2.7, 8.2.8, 8.2.19, 8.3.3, 8.3.27, 8.3.28, 8.3.29, 8.3.30, 8.3.31 added. Section 8.2.1, 8.3.1 changed.
2.9	11 May 2018	Valentin Tkachev	Section 2, 5, 5.1, 5.2, 5.3 changed.
2.10	7 June 2018	Valentin Tkachev	Section 5.1, 5.2, 5.3 changed.

2 Function overview

The signotec WebSocket Pad Server is signotec's solution for communicating with signotec LCD signature pads from a Web application without having to use a browser plug-in. The server must be run on the PC to which the signature device is connected. A Web application that runs in a browser can then communicate with the server via the WebSocket protocol and activate the connected signature device. Depending on the setup configuration, the `hosts` file is edited during installation of the server. See the 'Installation options' chapter for details. Depending on the setup configuration, the server can then be activated at `local.signotecwebsocket.de`, `localhost` or at `127.0.0.1` on port 49494 (by default) as follows using JavaScript:

```
websocket = new WebSocket("wss://local.signotecwebsocket.de:49494");
```

```
websocket = new WebSocket("wss://localhost:49494");
```

```
websocket = new WebSocket("wss://127.0.0.1:49494");
```

The port can be configured as a parameter value when the server starts. The server can therefore be started as follows, for example:

```
STPadServer.exe 49494
```

Communication with the server takes place with SSL encryption via the WebSocket protocol using JavaScript Object Notation (JSON).

3 System requirements for Windows

The signotec WebSocket Pad Server can be run under all Windows operating systems from Windows XP. The WebSocket protocol is supported by the following browsers, among others:

- Google Chrome (from Version 4.0.249.0)
- Mozilla Firefox (from Version 24.0)
- Opera (from Version 10.70)
- Microsoft Internet Explorer (from Version 10.0)
- Microsoft Edge

Information on the proxy server settings:

Please see the 'Installation options' chapter.

Information on SSL encryption:

The signotec WebSocket Pad Server uses its own SSL certificate in order to ensure that it can be called from Web applications that use SSL. This is set up automatically when the server is installed.

Information on the Transport Layer Security (TLS) protocol:

The lowest TLS version supported is 1.1. Please ensure that TLS 1.1 or a new version is enabled in the Windows Internet options ('Advanced' tab). If using Mozilla Firefox, change the settings as follows: Enter `about:config` in the address bar and search the `security.tls.version.max` or `security.tls.version.min` settings. Both entries must have a value of 1 or higher.

Information on use in Citrix environments:

The WebSocket Pad Server is not supported by Microsoft RDP sessions. If the WebSocket Pad Server is used in a Citrix environment in which there are multiple concurrent users, virtual IP loopback support must be set up for all used browsers and the WebSocket Pad Server.

Further information on setting up the virtual IP loopback can be found at:

<http://docs.citrix.com/en-us/xenapp-and-xendesktop/7-6/xad-deliver-virtual-ip.html>

Note that unfortunately, this configuration does not work in Internet Explorer:

<http://support.citrix.com/article/CTX122749>.

4 System requirements for Linux

The WebSocket protocol is supported by the following browsers, among others:

- Google Chrome (from Version 4.0.249.0)
- Mozilla Firefox (from Version 24.0)
- Opera (from Version 10.70)

Information on the proxy server settings:

Please see the 'Installation options' chapter.

Information on SSL encryption:

The signotec WebSocket Pad Server uses its own SSL certificate in order to ensure that it can be called from Web applications that use SSL. This requires adding the 'signotec GmbH CA' root certificate (root.crt) to the list of trusted root certification authorities.

Information on the Transport Layer Security (TLS) protocol:

The lowest TLS version supported is 1.1. Please ensure that TLS 1.1 or a new version is enabled in the Windows Internet options ('Advanced' tab). If using Mozilla Firefox, change the settings as follows: Enter `about:config` in the address bar and search the `security.tls.version.max` or `security.tls.version.min` settings. Both entries must have a value of 1 or higher.

5 Installation options

A setup program is available for Windows. The options for the Windows setup are described in sections 5.1, 5.2, 5.3, 5.4, 5.5, 5.6 and 5.7. If you are installing the WebSocket Pad Server under Linux, please see section 5.8.

One of the three options must be selected during installation. The selected option has an impact on the editing of system files and the URL, under which the signotec WebSocket Pad Server can be addressed.

5.1 Option 1 (local.signotecwebsocket.de)

When using this option, the signotec WebSocket Pad Server can only be addressed via the `local.signotecwebsocket.de` Connection-URL. This URL may be changed by a setup property, please refer to the chapters 5.6 and 5.7 for details.

Please note the following limitations:

Internet browser	local.signotecwebsocket.de	localhost	127.0.0.1
Microsoft Edge	supported	not supported	not supported
Microsoft Internet Explorer	supported	not supported	not supported
Mozilla Firefox	supported	supported	supported
Google Chrome	supported	not supported	not supported
Opera	supported	not supported	not supported

The following entry is made in the Windows `hosts` file during installation:

```
127.0.0.1    local.signotecwebsocket.de
```

The `hosts` file is located in the `%SystemRoot%\System32\drivers\etc` folder. This entry is removed during deinstallation.

When using a proxy server, this must be bypassed for the `local.signotecwebsocket.de` Connection-URL. For this reason, when the server is started, an exception is added to the Internet options for this DNS name and removed when it is ended. The entry is located in the `ProxyOverride` string in the registry under `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Internet Settings`.

5.2 Option 2 (local.signotecwebsocket.de, localhost and 127.0.0.1)

This option should typically always be used. When using this option, the signotec WebSocket Pad Server can be addressed via the `local.signotecwebsocket.de`, `localhost` or `127.0.0.1` Connection-URL. The first of this URLs may be changed by a setup property, please refer to the chapters 5.6 and 5.7 for details.

Please note the following limitations:

Internet browser	local.signotecwebsocket.de	localhost	127.0.0.1
Microsoft Edge	supported	supported	supported
Microsoft Internet Explorer	supported	supported	supported
Mozilla Firefox	supported	supported	supported
Google Chrome	supported	supported	supported
Opera	supported	supported	supported

All of the settings described in Option 1 must be specified.

When using a proxy server, it must be bypassed for local addresses in order to ensure that 127.0.0.1 can be used. This must be manually configured in the proxy settings of the Windows Internet options or Firefox settings.

5.3 Option 3 (localhost and 127.0.0.1)

This option should only be used if for any reason it must be possible to address the signotec WebSocket Pad Server via localhost or 127.0.0.1 only.

Please note the following limitations:

Internet browser	local.signotecwebsocket.de	localhost	127.0.0.1
Microsoft Edge	not supported	supported	supported
Microsoft Internet Explorer	not supported	supported	supported
Mozilla Firefox	not supported	supported	supported
Google Chrome	not supported	supported	supported
Opera	not supported	supported	supported

The Windows hosts file is not changed if this option is used.

When using a proxy server, it must be bypassed for local addresses. This must be manually configured in the proxy settings of the Windows Internet options or Firefox settings.

5.4 Enable Edge support (allow loopback)

By default, Edge does not allow localhost connections. The setup file contains the "ALLOW_EDGE_LOOPBACK" setup property for loopback activation. The permitted values for the setup property are "Yes" to enable localhost connections and no value ("") to disable localhost connections. This property can either be silent or enabled/disabled by ticking or unticking the 'Enable Edge support (allow loopback)' box in the GUI. localhost connections are permitted by default.

Further information on Edge localhost connections can be found at:
<https://msdn.microsoft.com/en-us/library/windows/apps/Hh780593.aspx>

5.5 Remove the old CA certificate from the Trusted Root Certification Authorities list

It is possible that when upgrading to the latest version, the old CA certificate with the SHA1 "DD:6C:94:8B:B6:C1:78:DA:AF:28:BD:80:01:4E:41:2A:23:EF:F5:67" was not removed.

The WebSocket Pad Server will try to remove the certificate at startup. If you do not want to remove the certificate, you should use the "DONT_REMOVE_OLD_CA_CERT" setup property with the value "Yes" during installation. By default, the property has the value "No".

5.6 Configure the Connection-URL

From version 1.3.2 it is possible to freely configure the Connection-URL. For this purpose, a setup property "WEBSOCKET_URL" is provided. For details, please refer to the chapter 5.7.

5.7 Silent installation

A silent installation makes it possible for you to install the WebSocket Pad Server without displaying dialog boxes, thus requiring no additional user interaction. To perform a silent installation, start the command prompt. Then switch to the folder containing the WebSocket Pad Server setup file.

CERTIFICATE_SELECTION options	Explanation
"Default"	Installation with option 1 – default value
"DefaultAndLocalhost"	Installation with option 2
"Localhost"	Installation with option 3
WEBSOCKET_URL options	Explanation
"url"	Installation sets up the WebSocket Pad Server for connection through the url
ALLOW_EDGE_LOOPBACK options	Explanation
"Yes"	Edge localhost connections permitted
""	Edge localhost connections not permitted
DONT_REMOVE_OLD_CA_CERT options	Explanation
"Yes"	Certificate is not removed
""	Certificate is removed
Feature variable	Explanation
Documentation	Installation of the documentation
Example	Installation of the example program

The variables are case-sensitive.

Examples:

```
signotec_WebSocket_Pad_Server_X.X.X.exe /s /v"/qn
CERTIFICATE_SELECTION=\\\"Default\\\""
```

```
signotec_WebSocket_Pad_Server_X.X.X.exe /s /v"/qn
CERTIFICATE_SELECTION=\\\"Localhost\\\""
```

```
signotec_WebSocket_Pad_Server_X.X.X.exe /s /v"ADDLOCAL=ALL REMOVE=Example /qn"
```

```
signotec_WebSocket_Pad_Server_X.X.X.exe /s /v"/qn ALLOW_EDGE_LOOPBACK=\\\"Yes\\\""
```

```
signotec_WebSocket_Pad_Server_X.X.X.exe /s /v"/qn ALLOW_EDGE_LOOPBACK=\\\"\\\""
```

```
signotec_WebSocket_Pad_Server_X.X.X.exe /s /v"/qn ALLOW_EDGE_LOOPBACK=\\\"\\\"
DONT_REMOVE_OLD_CA_CERT=\\\"Yes\\\""
```

```
signotec_WebSocket_Pad_Server_X.X.X.exe /s /v"/qn
WEBSOCKET_URL=\\\"test.websocket.url.com\\\" ALLOW_EDGE_LOOPBACK=\\\"\\\"
DONT_REMOVE_OLD_CA_CERT=\\\"Yes\\\""
```

5.8 Installation under Linux

The WebSocket Pad Server consists of the following files:

libSTCPIImageEngine.so

libSTPadLib.so

WSSTPadServer

The WebSocket Pad Server has the following dependencies:

libpng.so

libtiff.so

libX11.so

libusb-1.0.so

libpthread.so

libboost_system.so

libboost_thread.so

libboost_locale.so

libssl.so

libcrypto.so

Before a connection can be set up between the Web browser and the WebSocket Pad Server, the root.crt certificate must be added to the list of trusted root certification authorities.

The SSL certificate server.pem and its private key serverkey.pem must be located in the same folder with the WSSTPadServer executable file.

The options described in sections 5.1, 5.2 and 5.3 are also valid for the installation of the WebSocket Pad Server under Linux.

Depending on the SSL certificates used (sections 5.1, 5.2 and 5.3), a mapping must be added to the `hosts` file:

127.0.0.1 local.signotecwebsocket.de

When using HID or WinUSB devices, libusb 1.0.16 or higher is also required, which can be downloaded free of charge from <http://www.libusb.org>.

The authorisations for libusb must be adjusted if necessary. For this, the MODE for USB must be changed to 0666:

```
# libusb device nodes
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", MODE="0666"
```

udev must be restarted afterwards:

```
udevadm control --reload-rules
```

In Debian, this setting is located in the file `/lib/udev/rules.d/50-udev-default.rules` or `/lib64/udev/rules.d/50-udev-default.rules`. The setting may be in a different location when using other distributions.

6 Local connection type of pads

It is possible to customise the configuration file installed locally on the client to search for pads by connection type. This then overwrites the configuration provided by the Web application with the command "TOKEN_CMD_SEARCH_FOR_PADS" or "TOKEN_CMD_API_DEVICE_SET_COM_PORT". This feature works with all types of connections. In Windows, there is a configuration file valid for all machines (practical option when operating terminal services) and a configuration file valid for all users.

6.1 User-specific configuration file for connection type

There is an optional configuration file that the user can use to set the type of connection to look for when searching for pads. The file should be called %APPDATA%\signotec\STPadServer\PadConfig.ini. It will look like this:

```
[CurrentUser]
USB=
SerialPorts=
IPs=
```

The following values are valid for the relevant keys:

Key	Values	Description
USB	"Yes"	A search will be made for HID and WinUSB devices.
	"No"	A search will not be made for HID and WinUSB devices.
SerialPorts	"None"	A search will not be made on the COM ports or virtual channels for devices.
	"All"	A search will be made for devices on all COM ports and/or virtual channels; this search may take an extremely long time depending on the hardware configuration.
	other	List of numbers separated by a semicolon; a search will be made on each COM port and/or virtual channel with the specified numbers; this search may take an extremely long time if no signotec device is connected to the port.
IPs	"None"	A search will not be made on the IP addresses for devices.
	other	List of addresses separated by a semicolon with the structure "IP4 address:Port" or "Host name:Port"; an Alpha or Delta with an Ethernet connection or an HID device that is connected to a signotec Ethernet USB adapter will be searched for at all specified IP addresses; <Address> is the IP address of the device and has the format x.x.x.x or the host name, <Port> is the port via which the signature device should be activated, generally 1002. A valid value would therefore be, for example, "192.168.100.100:1002" or "host1:1002"; for details about the signotec Ethernet USB adapter, please refer to your contact at signotec.

If this file exists and it contains at least one validly configured connection type, the WebSocket Pad Server will search on this port or these ports for pads, regardless of what the

Web application has defined with the command "TOKEN_CMD_SEARCH_FOR_PADS" or "TOKEN_CMD_API_DEVICE_SET_COM_PORT".

6.2 System-wide configuration file for connection type

There is an optional configuration file that an administrator can use to select the type of connection for all or only specific users to look for when searching for pads. The file should be called %PROGRAMDATA%\signotec\STPadServer\PadConfig.ini. While the file should have the format as seen above, several sections may be used according to the following schema:

Section	Description
[<Computer name>]	If the computer name matches one of the available sections, the following keys are valid, regardless of which user is logged on. All other sections are ignored. (This is a useful option for terminal server environments and IP pads where the pad's IP address is not determined based on the user who is signed on but on the client on which the user is currently working. When choosing this option, the 'CLIENTNAME' environment variable on the terminal server must contain the name of the client computer.)
[<User name>]	If the user name matches one of the available sections, the following keys are valid. Any existing [CurrentUser] section is ignored.
[CurrentUser]	The following keys are valid for all users who do not meet one of the criteria listed above.

If this file exists and it contains at least one validly configured connection type, the WebSocket Pad Server will search on this port or these ports for pads, regardless of the value set in %APPDATA%\signotec\STPadServer\PadConfig.ini or what the Web application has defined with the command "TOKEN_CMD_SEARCH_FOR_PADS" or "TOKEN_CMD_API_DEVICE_SET_COM_PORT".

6.3 Installation of PadConfig.ini

The optional system-wide configuration file PadConfig.ini is installed during the setup process under %PROGRAMDATA%\signotec\STPadServer\ if it is located next to the setup file during installation. This file receives the default privileges of the directory %PROGRAMDATA%. It can therefore only be modified by administrators. If the file is not next to the setup file, no files are installed. When the file is uninstalled, any existing file as well as the folder above it (if empty) are deleted. The file is not deleted during an update. It is overwritten while an update is being installed if another file is next to the setup file.

7 Standard and API mode

The WebSocket Server can be used in two modes. The standard mode offers an easy-to-use interface with a restricted scope of functions. In API mode, signotec LCD signature pads can be flexibly integrated into a Web application of the customer.

7.1 Standard mode

In standard mode, the functions provided by the WebSocket Server are sufficient for capturing a signature with a signature device. A given dialog can be displayed on a signature device, and the signature can be captured in a format for real-time display, in SignData format and as an image. Integration into a customer Web application using this mode is simple and requires little time and effort.

7.2 API mode

In API mode, the functions provided by the WebSocket Server are similar to those of signoPAD API Windows. An explanation of the syntax is provided below. For information on the precise functionality of the individual commands, please refer to the signoPAD API documentation.

This mode is currently not supported under Linux.

In API mode, all values of the "TOKEN_CMD" key begin with "TOKEN_CMD_API_" and then contain the method name, for example, "TOKEN_CMD_API_DEVICE_GET_COUNT" in order to search for connected devices with the DeviceGetCount() method from the signoPAD API. In relation to the parameters, please note the following special features:

7.2.1 Numbers

Numbers must be passed as strings. The point must be used as the decimal separator for floating point numbers. Hexadecimal numbers can be passed by placing "0x" in front; they can, of course, also be passed as decimal numbers.

Numbers – examples

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DISPLAY_SET_PDF", "TOKEN_PARAM_X_POS": "0",
"TOKEN_PARAM_Y_POS": "0", "TOKEN_PARAM_PAGE": "1", "TOKEN_PARAM_SCALE":
"1.0", "TOKEN_PARAM_OPTIONS": "0x01" }

{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_DISPLAY_SET_PDF", "TOKEN_PARAM_RETURNCODE": "0" }
```

7.2.2 Enums

Numbers must be passed instead of enum values.

Passing an enum value – example

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DISPLAY_SET_TEXT", "TOKEN_PARAM_X_POS": "0",
"TOKEN_PARAM_Y_POS": "0", "TOKEN_PARAM_ALIGNMENT": "0",
"TOKEN_PARAM_TEXT": "Please sign here:"}

{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_DISPLAY_SET_TEXT", "TOKEN_PARAM_RETURNCODE": "60"}
```

7.2.3 Boolean parameters

Boolean parameters must be passed as "TRUE" or "FALSE".

Boolean parameter – example

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_SENSOR_SET_PEN_SCROLLING_ENABLED", "TOKEN_PARAM_ENABLE":
"TRUE"}

{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_SENSOR_SET_PEN_SCROLLING_ENABLED",
"TOKEN_PARAM_RETURNCODE": "0"}
```

7.2.4 Colours

Parameters that define colours must be passed as a number representing an RGB value. The last-but-two byte determines the share of the red channel, the last-but-one byte the share of the green channel and the last byte determines the share of the blue channel. Therefore, the colour red corresponds to "0xFF0000" or "16711680", the colour green to "0x00FF00" or "65280" and the colour blue to "0x0000FF" or "255".

7.2.5 Pointer parameters

Pointers cannot be used. Instead, the data can be passed directly with the request. The data changed by the method is delivered in the response.

Pointer as an in parameter – example

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DEVICE_SET_COM_PORT", "TOKEN_PARAM_PORT_LIST":
"HID;1;4;IP=192.168.100.100:1002"}

{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_DEVICE_SET_COM_PORT", "TOKEN_PARAM_RETURNCODE": "0"}
```

Pointer as an out parameter – example

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DEVICE_GET_INFO", "TOKEN_PARAM_INDEX": "0"}

{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_DEVICE_GET_INFO", "TOKEN_PARAM_SERIAL": "1500001234",
"TOKEN_PARAM_TYPE": "15"}
```

7.2.6 Byte arrays and images

Instead of byte arrays and images, Base64 strings must be passed and returned. The PNG format is recommended for images.

Passing a PNG image – example

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DISPLAY_SET_IMAGE", "TOKEN_PARAM_X_POS": "0",
"TOKEN_PARAM_Y_POS": "0", "TOKEN_PARAM_BITMAP":
"iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAIAAACQd1PeAAAAAXNSR0IArs4c6QAA
AARnQU1BAACxjwv8YQUAAAJcEhZcwAADSMAAA7DAdvqGQAAAAMSURVBhXY/j/
/z8ABf4C/qc1gYQAAAAASUVORK5CYII="}

{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_DISPLAY_SET_IMAGE", "TOKEN_PARAM_RETURNCODE": "0" }
```

Return of a byte array – example

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA" }

{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA", "TOKEN_PARAM_SIGN_DATA":
"CgAAAAEAAAABvAAgCVQVVBVUVABAEAAEAQAABA2IYalx2GVVCVU4WzHpbduCrrInm
GTLTbzfoOd/WFQo47vAi9Af4AFc=" }
```

7.2.7 Files

Files (for example, images from the local hard disk) are not supported. Data can only be exchanged as Base64-encoded strings.

8 Communication setup

Communication with the WebSocket server is bidirectional and asynchronous. There are some instructions that expect a response and some instructions that do not expect a response. Each instruction is a JSON string that contains an object with different key/value pairs.

The WebSocket Server can be used in two modes. The standard mode offers an easy-to-use interface with a restricted scope of functions. In API mode, the WebSocket Server provides the same scope of functions as the STPadLib.dll component from signoPAD API Windows. In this mode, signotec LCD signature pads can be flexibly integrated into a Web application of the customer.

8.1 Types (TOKEN_TYPE key)

Available from server version 1.0.0.0 onwards.

Each instruction must contain a "TOKEN_TYPE" key that defines the type of instruction.

8.1.1 Requests (TOKEN_TYPE_REQUEST value)

Available from server version 1.0.0.0 onwards.

The "TOKEN_TYPE_REQUEST" value defines the instruction as a request from the Web application to the WebSocket server. The request must contain a command and may contain one or more parameters. A request is always answered by an instruction of type "TOKEN_TYPE_RESPONSE". The Web application should always wait for the response before it sends a new request.

The combining of commands from the standard mode with commands from the API mode can lead to unexpected behaviour and should therefore be avoided.

The following commands are currently defined for both modes:

- "TOKEN_CMD_SET_INTERFACE_VERSION"
- "TOKEN_CMD_GET_SERVER_VERSION"

The following commands are currently defined for the standard mode:

- "TOKEN_CMD_SEARCH_FOR_PADS"
- "TOKEN_CMD_OPEN_PAD"
- "TOKEN_CMD_SELECTION_DIALOG"
- "TOKEN_CMD_SIGNATURE_START"
- "TOKEN_CMD_SIGNATURE_RETRY"
- "TOKEN_CMD_SIGNATURE_STOP"
- "TOKEN_CMD_SIGNATURE_CONFIRM"
- "TOKEN_CMD_SIGNATURE_CANCEL"
- "TOKEN_CMD_SIGNATURE_SIGN_DATA"
- "TOKEN_CMD_SIGNATURE_IMAGE"
- "TOKEN_CMD_CLOSE_PAD"

The following commands are currently defined for the API mode:

- "TOKEN_CMD_API_DEVICE_SET_COM_PORT"
- "TOKEN_CMD_API_DEVICE_GET_CONNECTION_TYPE"
- "TOKEN_CMD_API_DEVICE_GET_COM_PORT"

- "TOKEN_CMD_API_DEVICE_GET_IP_ADDRESS"
- "TOKEN_CMD_API_DEVICE_GET_COUNT"
- "TOKEN_CMD_API_DEVICE_GET_INFO"
- "TOKEN_CMD_API_DEVICE_GET_VERSION"
- "TOKEN_CMD_API_DEVICE_OPEN"
- "TOKEN_CMD_API_DEVICE_CLOSE"
- "TOKEN_CMD_API_SENSOR_GET_SAMPLE_RATE_MODE"
- "TOKEN_CMD_API_SENSOR_SET_SAMPLE_RATE_MODE"
- "TOKEN_CMD_API_SENSOR_SET_SIGN_RECT"
- "TOKEN_CMD_API_SENSOR_CLEAR_SIGN_RECT"
- "TOKEN_CMD_API_SENSOR_ADD_HOT_SPOT"
- "TOKEN_CMD_API_SENSOR_SET_HOT_SPOT_MODE"
- "TOKEN_CMD_API_SENSOR_CLEAR_HOT_SPOTS"
- "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION"
- "TOKEN_CMD_API_SIGNATURE_START"
- "TOKEN_CMD_API_SIGNATURE_STOP"
- "TOKEN_CMD_API_SIGNATURE_CONFIRM"
- "TOKEN_CMD_API_SIGNATURE_RETRY"
- "TOKEN_CMD_API_SIGNATURE_CANCEL"
- "TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA"
- "TOKEN_CMD_API_SIGNATURE_SAVE_AS_STREAM_EX"
- "TOKEN_CMD_API_SIGNATURE_GET_BOUNDS"
- "TOKEN_CMD_API_SIGNATURE_GET_STATE"
- "TOKEN_CMD_API_DISPLAY_ERASE"
- "TOKEN_CMD_API_DISPLAY_ERASE_RECT"
- "TOKEN_CMD_API_DISPLAY_CONFIG_PEN"
- "TOKEN_CMD_API_DISPLAY_SET_FONT"
- "TOKEN_CMD_API_DISPLAY_SET_FONT_COLOR"
- "TOKEN_CMD_API_DISPLAY_SET_TARGET"
- "TOKEN_CMD_API_DISPLAY_SET_TEXT"
- "TOKEN_CMD_API_DISPLAY_SET_TEXT_IN_RECT"
- "TOKEN_CMD_API_DISPLAY_SET_IMAGE"
- "TOKEN_CMD_API_DISPLAY_SET_IMAGE_FROM_STORE"
- "TOKEN_CMD_API_DISPLAY_SAVE_IMAGE_AS_STREAM"
- "TOKEN_CMD_API_DISPLAY_SET_STANDBY_IMAGE"
- "TOKEN_CMD_API_DISPLAY_CONFIG_SLIDE_SHOW_EX"
- "TOKEN_CMD_API_DISPLAY_GET_STANDBY_ID"
- "TOKEN_CMD_API_DISPLAY_GET_WIDTH"
- "TOKEN_CMD_API_DISPLAY_GET_HEIGHT"
- "TOKEN_CMD_API_DISPLAY_SET_ROTATION"
- "TOKEN_CMD_API_DISPLAY_GET_ROTATION"

8.1.2 Responses (TOKEN_TYPE_RESPONSE value)

Available from server version 1.0.0.0 onwards.

The "TOKEN_TYPE_RESPONSE" value defines an instruction as a response from the WebSocket server to a request of type "TOKEN_TYPE_REQUEST" made by the Web application. A response always contains a command and one or more parameters. The "TOKEN_CMD_ORIGIN" key always corresponds to the value of the "TOKEN_CMD" key from the request, see also "TOKEN_TYPE_REQUEST".

8.1.3 Events (TOKEN_TYPE_SEND value)

Available from server version 1.0.0.0 onwards.

The "TOKEN_TYPE_SEND" value defines an instruction as an event that is triggered by the server. An event always contains a command and may contain one or more parameters. An event does not expect a response.

The following events are currently defined for both modes:

- "TOKEN_CMD_SIGNATURE_POINT"
- "TOKEN_CMD_DISCONNECT"

The following events are currently defined for the standard mode:

- "TOKEN_CMD_SELECTION_CHANGE"
- "TOKEN_CMD_SELECTION_CONFIRM"
- "TOKEN_CMD_SELECTION_CANCEL"
- "TOKEN_CMD_SIGNATURE_RETRY"
- "TOKEN_CMD_SIGNATURE_CONFIRM"
- "TOKEN_CMD_SIGNATURE_CANCEL"
- "TOKEN_CMD_ERROR"

The following events are currently defined for the API mode:

- "TOKEN_CMD_API_SENSOR_HOT_SPOT_PRESSED"

8.2 Commands (TOKEN_CMD and TOKEN_CMD_ORIGIN keys)

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_CMD" key defines the command that should be executed for requests and events. The value of the "TOKEN_CMD_ORIGIN" key in instructions of type "TOKEN_TYPE_RESPONSE" always corresponds to the value of the "TOKEN_CMD" key from the request.

8.2.1 TOKEN_CMD_SET_INTERFACE_VERSION value

Available from server version 1.0.2.4 onwards.

A request with the "TOKEN_CMD_SET_INTERFACE_VERSION" command sets the version of the server interface to be used. At each key is documented, from which version it or one of its parameters are available. It is recommended that you always set the server version for which development is taking place, in order to ensure that the application functions correctly when a newer version of the WebSocket Pad Server is installed on the client at a later point.

For example, server versions 1.0.0.0 onwards support version 1.0.0.0 of the server interface, while server versions 1.0.2.4 onwards support version 1.0.2.4 of the server interface.

This value has the following keys as a parameter:

- "TOKEN_PARAM_VERSION"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"

- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SET_INTERFACE_VERSION", "TOKEN_PARAM_VERSION": "1.0.2.4" }
```

8.2.2 TOKEN_CMD_GET_SERVER_VERSION value

Available from server version 1.0.2.4 onwards.

A request with the "TOKEN_CMD_GET_SERVER_VERSION" command delivers the server version. This information can be used to make the behaviour of the Web application dependent on the server version available.

As a request, this value has no parameters.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_VERSION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_GET_SERVER_VERSION" }
```

8.2.3 TOKEN_CMD_SEARCH_FOR_PADS value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SEARCH_FOR_PADS" command starts a search for connected signature devices. If devices are found, their properties are determined and returned in the response.

Under Linux, only HID pads are found.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_PAD_SUBSET"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_CONNECTED_PADS"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SEARCH_FOR_PADS", "TOKEN_PARAM_PAD_SUBSET":  
  "HID;1;4;IP=192.168.100.100:1002" }
```

8.2.4 TOKEN_CMD_OPEN_PAD value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_OPEN_PAD" command opens a connection to a device.

As a request, this value must have the following keys as a parameter:

- "TOKEN_PARAM_PAD_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_PAD_DISPLAY_WIDTH"
- "TOKEN_PARAM_PAD_DISPLAY_HEIGHT"
- "TOKEN_PARAM_PAD_SAMPLING_RATE"
- "TOKEN_PARAM_PAD_X_RESOLUTION" (available from server version 1.0.2.4 onwards).
- "TOKEN_PARAM_PAD_Y_RESOLUTION" (available from server version 1.0.2.4 onwards).

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD": "TOKEN_CMD_OPEN_PAD",
  "TOKEN_PARAM_PAD_INDEX": "0" }
```

Example for the answer from server version 1.0.0.0 onwards:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
  "TOKEN_CMD_OPEN_PAD", "TOKEN_PARAM_RETURN_CODE": "0",
  "TOKEN_PARAM_PAD_DISPLAY_WIDTH": "640", "TOKEN_PARAM_PAD_DISPLAY_HEIGHT":
  "480", "TOKEN_PARAM_PAD_SAMPLING_RATE": "250" }
```

Example for the answer from server version 1.0.2.4 onwards:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
  "TOKEN_CMD_OPEN_PAD", "TOKEN_PARAM_RETURN_CODE": "0",
  "TOKEN_PARAM_PAD_DISPLAY_WIDTH": "640", "TOKEN_PARAM_PAD_X_RESOLUTION":
  "1920", "TOKEN_PARAM_PAD_DISPLAY_HEIGHT": "480",
  "TOKEN_PARAM_PAD_Y_RESOLUTION": "1440", "TOKEN_PARAM_PAD_SAMPLING_RATE":
  "250" }
```

8.2.5 TOKEN_CMD_SELECTION_DIALOG value

Available from server version 1.4.0.0 onwards.

A request with the "TOKEN_CMD_SELECTION_DIALOG" command displays a dialog with freely configurable selection fields as well as a 'Cancel' and a 'Confirm' button, provided a connection has been opened beforehand using the "TOKEN_CMD_OPEN_PAD" command. The selection fields are positioned dynamically depending on the text length and the size of the screen of the signature device and, if necessary, split over several pages. If the user changes the value of a selection field by clicking, an event of the type "TOKEN_CMD_SELECTION_CHANGE" is triggered. If the 'Cancel' button is pressed, an event of the type "TOKEN_CMD_SELECTION_CANCEL" is triggered. If the 'Confirm' button on the last page is pressed, an event of the type "TOKEN_CMD_SELECTION_CONFIRM" will be triggered. If the 'Confirm' button on a previous page is pressed, the next page is displayed without informing the application. If an error occurs, an event of type "TOKEN_CMD_ERROR" will be triggered.

The request with the "TOKEN_CMD_SELECTION_DIALOG" command is not implemented under Linux and returns the error -25.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_FIELD_LIST"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SELECTION_DIALOG", "TOKEN_PARAM_FIELD_LIST":
[{"TOKEN_PARAM_FIELD_ID": "Check1", "TOKEN_PARAM_FIELD_TEXT": " I agree
to the processing of my data.", "TOKEN_PARAM_FIELD_CHECKED": "FALSE",
"TOKEN_PARAM_FIELD_REQUIRED": "TRUE"}, {"TOKEN_PARAM_FIELD_ID": "Check2",
"TOKEN_PARAM_FIELD_TEXT": "I would like to subscribe to the newsletter.",
"TOKEN_PARAM_FIELD_CHECKED": "TRUE", "TOKEN_PARAM_FIELD_REQUIRED":
"FALSE"}]}
```

8.2.6 TOKEN_CMD_SELECTION_CHANGE value

Available from server version 1.4.0.0 onwards.

An event of type "TOKEN_CMD_SELECTION_CHANGE" is triggered if the user clicks a field of the selection dialog on the signature device with the pen.

This value can have the following keys as a parameter:

- "TOKEN_PARAM_FIELD_ID"
- "TOKEN_PARAM_FIELD_CHECKED"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":
"TOKEN_CMD_SELECTION_CHANGE", "TOKEN_PARAM_FIELD_ID": "Check1",
"TOKEN_PARAM_FIELD_CHECKED": "TRUE" }
```

8.2.7 TOKEN_CMD_SELECTION_CONFIRM value

Available from server version 1.4.0.0 onwards.

An event of type "TOKEN_CMD_SELECTION_CONFIRM" is triggered if the user clicks the 'Confirm' button on the last page of a selection dialog on the signature device with the pen. The typical reaction to this event should be a request with the "TOKEN_CMD_SIGNATURE_START" or "TOKEN_CMD_CLOSE_PAD" command.

This value no parameters.

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":
"TOKEN_CMD_SELECTION_CONFIRM" }
```

8.2.8 TOKEN_CMD_SELECTION_CANCEL value

Available from server version 1.4.0.0 onwards.

An event of type "TOKEN_CMD_SELECTION_CANCEL" is triggered if the user clicks the 'Cancel' button of a selection dialog on the signature device with the pen. The typical reaction to this event should be a request with the "TOKEN_CMD_CLOSE_PAD" command.

This value no parameters.

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":  
  "TOKEN_CMD_SELECTION_CANCEL" }
```

8.2.9 TOKEN_CMD_SIGNATURE_START value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_START" command starts the signature capture process, provided a connection has been opened beforehand using the "TOKEN_CMD_OPEN_PAD" command.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_FIELD_NAME"
- "TOKEN_PARAM_CUSTOM_TEXT"
- "TOKEN_PARAM_PAD_ENCRYPTION" (available from server version 1.3.0.0 onwards)
- "TOKEN_PARAM_DOCHASH" (available from server version 1.3.0.0 onwards)
- "TOKEN_PARAM_ENCRYPTION_CERT" (available from server version 1.3.0.0 onwards)

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Examples:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SIGNATURE_START", "TOKEN_PARAM_FIELD_NAME": "Customer  
  Signature", "TOKEN_PARAM_CUSTOM_TEXT": "Please sign here!" }
```

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST",  
  "TOKEN_CMD": "TOKEN_CMD_SIGNATURE_START",  
  "TOKEN_PARAM_FIELD_NAME": "Customer Signature",  
  "TOKEN_PARAM_CUSTOM_TEXT": "Please sign!",  
  "TOKEN_PARAM_PAD_ENCRYPTION": "TRUE",  
  "TOKEN_PARAM_DOCHASH": "AAECAwQFBgcICQoLDA0ODxAREhMUFRYXGBkaGxwdHh8=",  
  "TOKEN_PARAM_ENCRYPTION_CERT": "MIICqTCCAZGgAwIBAgIBATANBgkqhkiXylbpSA==" }
```

8.2.10 TOKEN_CMD_SIGNATURE_RETRY value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_RETRY" command discards the signature data without ending the signature capture process and deletes the rendered signature from the

signature device's display. This command will start a new capture process if the signature capture process was terminated beforehand using the "TOKEN_CMD_SIGNATURE_STOP" command.

An event with the "TOKEN_CMD_SIGNATURE_RETRY" command will be triggered if the user clicks the 'Retry' button on the signature device with the pen. The typical reaction to this event should be a request with the "TOKEN_CMD_SIGNATURE_RETRY" command.

This value has no parameters either as a request or as an event.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SIGNATURE_RETRY" }
```

8.2.11 TOKEN_CMD_SIGNATURE_STOP value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_STOP" command terminates the signature capture process that is currently running and caches the captured signature data.

As a request, this value has no parameters.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SIGNATURE_STOP" }
```

8.2.12 TOKEN_CMD_SIGNATURE_CONFIRM value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_CONFIRM" command terminates the signature capture process that is currently running, caches the captured signature data and, in contrast to "TOKEN_CMD_SIGNATURE_STOP", erases the entire device LCD.

An event with the "TOKEN_CMD_SIGNATURE_CONFIRM" command will be triggered if the user clicks the 'Confirm' button on the signature device with the pen. The typical reaction to this event should be a request with the "TOKEN_CMD_SIGNATURE_STOP" or "TOKEN_CMD_SIGNATURE_CONFIRM" command.

This value has no parameters either as a request or as an event.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

The value of the "TOKEN_PARAM_RETURN_CODE" key when the "TOKEN_CMD_SIGNATURE_CONFIRM" command has been called successfully contains the number of captured signature points.

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SIGNATURE_CONFIRM" }
```

Example answer:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":  
  "TOKEN_CMD_SIGNATURE_CONFIRM", "TOKEN_PARAM_RETURN_CODE": "276" }
```

8.2.13 TOKEN_CMD_SIGNATURE_CANCEL value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_CANCEL" command terminates the signature capture process that is currently running, discards the signature data and erases the entire device LCD.

An event with the "TOKEN_CMD_SIGNATURE_CANCEL" command will be triggered if the user clicks the 'Cancel' button on the signature device with the pen. The typical reaction to this event should be a request with the "TOKEN_CMD_SIGNATURE_CANCEL" command.

This value has no parameters either as a request or as an event.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SIGNATURE_CANCEL" }
```

8.2.14 TOKEN_CMD_SIGNATURE_SIGN_DATA value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_SIGN_DATA" command retrieves the SignData of a captured signature from the device. SignData is an encrypted, compressed, biometric data format that can be stored in a database or as a tag in a TIFF or PDF document.

As a request, this value has no parameters.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_SIGNATURE_SIGN_DATA"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SIGNATURE_SIGN_DATA" }
```

8.2.15 TOKEN_CMD_SIGNATURE_IMAGE value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_IMAGE" command returns a captured signature as image data. The image is created with the aspect ratio of the rectangle that surrounds the signature.

The request with the "TOKEN_CMD_SIGNATURE_IMAGE" command is not implemented under Linux and returns the error -25.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_FILE_TYPE"
- "TOKEN_PARAM_PEN_WIDTH"; available from server version 1.1.0.0 onwards.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_FILE"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":  
  "TOKEN_CMD_SIGNATURE_IMAGE", "TOKEN_PARAM_FILE_TYPE": "1",  
  "TOKEN_PARAM_PEN_WIDTH": "0" }
```

8.2.16 TOKEN_CMD_CLOSE_PAD value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_CLOSE_PAD" command closes the connection to a device. Before closing, a currently running signature capture process is terminated. Captured signature data is discarded.

As a request, this value must have the following keys as a parameter:

- "TOKEN_PARAM_PAD_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD": "TOKEN_CMD_CLOSE_PAD",  
  "TOKEN_PARAM_PAD_INDEX": "0" }
```

8.2.17 TOKEN_CMD_SIGNATURE_POINT value

Available from server version 1.0.0.0 onwards.

An event of type "TOKEN_CMD_SIGNATURE_POINT" is triggered for each captured point during the signing process. The Web application can use this event to render the signature in real time.

This value has the following keys as a parameter:

- "TOKEN_PARAM_POINT"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":  
  "TOKEN_CMD_SIGNATURE_POINT", "TOKEN_PARAM_POINT": { "x": "218", "y":  
    "245", "p": "0" } }
```

8.2.18 TOKEN_CMD_DISCONNECT value

Available from server version 1.0.0.0 onwards.

An event of type "TOKEN_CMD_DISCONNECT" is triggered as soon as the connection to a device is terminated via an external event (for example, the device is unplugged). The event will only be triggered for a device that has been opened beforehand using a request with the "TOKEN_CMD_OPEN_PAD" or "TOKEN_CMD_API_DEVICE_OPEN" command.

This value has the following keys as a parameter:

- "TOKEN_PARAM_PAD_INDEX"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD": "TOKEN_CMD_DISCONNECT", "  
  TOKEN_PARAM_PAD_INDEX": "0" }
```

8.2.19 TOKEN_CMD_ERROR value

Available from server version 1.4.0.0 onwards.

An event of type "TOKEN_CMD_ERROR" is triggered if an error occurs outside of an request.

This value can have the following keys as a parameter:

- "TOKEN_PARAM_ERROR_CONTEXT"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD": "TOKEN_CMD_ERROR",  
  "TOKEN_PARAM_ERROR_CONTEXT": "TOKEN_CMD_SELECTION_DIALOG",  
  "TOKEN_PARAM_RETURN_CODE": "-400", "TOKEN_PARAM_ERROR_DESCRIPTION": "Not  
  all mandatory fields have been selected." }
```

8.2.20 TOKEN_CMD_API_DEVICE_SET_COM_PORT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_PORT_LIST"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceSetComPort() method in the signoPAD API.

8.2.21 TOKEN_CMD_API_DEVICE_GET_CONNECTION_TYPE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetConnectionType() method in the signoPAD API.

8.2.22 TOKEN_CMD_API_DEVICE_GET_COM_PORT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetComPort() method in the signoPAD API.

8.2.23 TOKEN_CMD_API_DEVICE_GET_IP_ADDRESS value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_IPADDRESS"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetIPAddress() method in the signoPAD API.

8.2.24 TOKEN_CMD_API_DEVICE_GET_COUNT value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetCount() method in the signoPAD API.

8.2.25 TOKEN_CMD_API_DEVICE_GET_INFO value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_SERIAL"
- "TOKEN_PARAM_TYPE"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetInfo() method in the signoPAD API.

8.2.26 TOKEN_CMD_API_DEVICE_GET_VERSION value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_VERSION"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetVersion() method in the signoPAD API.

8.2.27 TOKEN_CMD_API_DEVICE_OPEN value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"
- "TOKEN_PARAM_ERASE_DISPLAY"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceOpen() method in the signoPAD API.

8.2.28 TOKEN_CMD_API_DEVICE_CLOSE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceClose() method in the signoPAD API.

8.2.29 TOKEN_CMD_API_SENSOR_GET_SAMPLE_RATE_MODE value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorGetSampleRateMode() method in the signoPAD API.

8.2.30 TOKEN_CMD_API_SENSOR_SET_SAMPLE_RATE_MODE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_MODE"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SensorSetSampleRateMode()` method in the signoPAD API.

8.2.31 TOKEN_CMD_API_SENSOR_SET_SIGN_RECT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SensorSetSignRect()` method in the signoPAD API.

8.2.32 TOKEN_CMD_API_SENSOR_CLEAR_SIGN_RECT value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SensorClearSignRect()` method in the signoPAD API.

8.2.33 TOKEN_CMD_API_SENSOR_ADD_HOT_SPOT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SensorAddHotSpot()` method in the signoPAD API.

8.2.34 **TOKEN_CMD_API_SENSOR_SET_HOT_SPOT_MODE** value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_MODE"
- "TOKEN_PARAM_HOTSPOT_ID"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SensorSetHotSpotMode()` method in the signoPAD API.

8.2.35 **TOKEN_CMD_API_SENSOR_CLEAR_HOT_SPOTS** value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SensorClearHotSpots()` method in the signoPAD API.

8.2.36 **TOKEN_CMD_API_SENSOR_HOT_SPOT_PRESSED** value

Available from server version 1.1.0.0 onwards.

This value has the following keys as a parameter:

- "TOKEN_PARAM_HOTSPOT_ID"

For details and an example, please see the documentation of the `SensorHotSpotPressed()` event in the signoPAD API.

8.2.37 **TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION** value

Available from server version 1.1.0.0 onwards.

A request with the "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" command delivers the resolution of the signature data that is delivered in the event of the "TOKEN_CMD_SIGNATURE_POINT" type.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_PAD_X_RESOLUTION"
- "TOKEN_PARAM_PAD_Y_RESOLUTION"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
  "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" }
```

Reply example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
  "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION", "TOKEN_PARAM_RETURN_CODE": "0",
  "TOKEN_PARAM_PAD_X_RESOLUTION": "1920", "TOKEN_PARAM_PAD_Y_RESOLUTION":
  "1440" }
```

8.2.38 TOKEN_CMD_API_SIGNATURE_START value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureStart() method in the signoPAD API.

8.2.39 TOKEN_CMD_API_SIGNATURE_STOP value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureStop() method in the signoPAD API.

8.2.40 TOKEN_CMD_API_SIGNATURE_CONFIRM value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"

- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureConfirm() method in the signoPAD API.

8.2.41 TOKEN_CMD_API_SIGNATURE_RETRY value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureRetry() method in the signoPAD API.

8.2.42 TOKEN_CMD_API_SIGNATURE_CANCEL value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_ERASE"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureCancel() method in the signoPAD API.

8.2.43 TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_SIGN_DATA"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureGetSignData() method in the signoPAD API.

8.2.44 TOKEN_CMD_API_SIGNATURE_SAVE_AS_STREAM_EX value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RESOLUTION"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"
- "TOKEN_PARAM_FILE_TYPE"
- "TOKEN_PARAM_PEN_WIDTH"
- "TOKEN_PARAM_PEN_COLOR"
- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_IMAGE"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SignatureSaveAsStreamEx()` method in the signoPAD API.

8.2.45 TOKEN_CMD_API_SIGNATURE_GET_BOUNDS value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_RIGHT"
- "TOKEN_PARAM_BOTTOM"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SignatureGetBounds()` method in the signoPAD API.

8.2.46 TOKEN_CMD_API_SIGNATURE_GET_STATE value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `SignatureState` property in the signoPAD API.

8.2.47 TOKEN_CMD_API_DISPLAY_ERASE value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayErase() method in the signoPAD API.

8.2.48 TOKEN_CMD_API_DISPLAY_ERASE_RECT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayEraseRect() method in the signoPAD API.

8.2.49 TOKEN_CMD_API_DISPLAY_CONFIG_PEN value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_PEN_COLOR"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayConfigPen() method in the signoPAD API.

8.2.50 TOKEN_CMD_API_DISPLAY_SET_FONT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_NAME"
- "TOKEN_PARAM_SIZE"
- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetFont() method in the signoPAD API.

8.2.51 TOKEN_CMD_API_DISPLAY_SET_FONT_COLOR value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_FONT_COLOR"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetFontColor() method in the signoPAD API.

8.2.52 TOKEN_CMD_API_DISPLAY_SET_TARGET value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_TARGET"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetTarget() method in the signoPAD API.

8.2.53 TOKEN_CMD_API_DISPLAY_SET_TEXT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_X_POS"
- "TOKEN_PARAM_Y_POS"
- "TOKEN_PARAM_ALIGNMENT"
- "TOKEN_PARAM_TEXT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `DisplaySetText()` method in the signoPAD API.

8.2.54 `TOKEN_CMD_API_DISPLAY_SET_TEXT_IN_RECT` value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_LEFT"`
- `"TOKEN_PARAM_TOP"`
- `"TOKEN_PARAM_WIDTH"`
- `"TOKEN_PARAM_HEIGHT"`
- `"TOKEN_PARAM_ALIGNMENT"`
- `"TOKEN_PARAM_TEXT"`
- `"TOKEN_PARAM_OPTIONS"`

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

For details and an example, please see the documentation of the `DisplaySetTextInRect()` method in the signoPAD API.

8.2.55 `TOKEN_CMD_API_DISPLAY_SET_IMAGE` value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_X_POS"`
- `"TOKEN_PARAM_Y_POS"`
- `"TOKEN_PARAM_BITMAP"`

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

For details and an example, please see the documentation of the `DisplaySetImage()` method in the signoPAD API.

8.2.56 `TOKEN_CMD_API_DISPLAY_SET_IMAGE_FROM_STORE` value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_STORE_ID"`

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

For details and an example, please see the documentation of the `DisplaySetImageFromStore()` method in the signoPAD API.

8.2.57 **TOKEN_CMD_API_DISPLAY_SAVE_IMAGE_AS_STREAM** value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_FILE_TYPE"
- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_IMAGE"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `DisplaySaveImageAsStream()` method in the signoPAD API.

8.2.58 **TOKEN_CMD_API_DISPLAY_SET_STANDBY_IMAGE** value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_BITMAP"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `DisplaySetStandbyImage()` method in the signoPAD API.

8.2.59 **TOKEN_CMD_API_DISPLAY_CONFIG_SLIDE_SHOW_EX** value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_SLIDE_LIST"
- "TOKEN_PARAM_DURATION_LIST"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the `DisplayConfigSlideShowEx()` method in the signoPAD API.

8.2.60 TOKEN_CMD_API_DISPLAY_GET_STANDBY_ID value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_ID"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayGetStandbyID() method in the signoPAD API.

8.2.61 TOKEN_CMD_API_DISPLAY_GET_WIDTH value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayWidth property in the signoPAD API.

8.2.62 TOKEN_CMD_API_DISPLAY_GET_HEIGHT value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayHeight property in the signoPAD API.

8.2.63 TOKEN_CMD_API_DISPLAY_SET_ROTATION value

Available from server version 1.3.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_ROTATION"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayRotation property in the signoPAD API.

8.2.64 TOKEN_CMD_API_DISPLAY_GET_ROTATION value

Available from server version 1.3.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayRotation property in the signoPAD API.

8.3 Parameter

Some instructions may contain parameters. These are defined below.

8.3.1 TOKEN_PARAM_RETURN_CODE key

Available from server version 1.0.0.0 onwards.

The "TOKEN_PARAM_RETURN_CODE" key contains an error number as a value. "0" means that no error has occurred. A negative value indicates that an error occurred during the process. A positive value has different meanings depending on the command. If the value is negative, the response can also contain the "TOKEN_PARAM_ERROR_DESCRIPTION" key.

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":  
  "TOKEN_CMD_SEARCH_FOR_PADS", "TOKEN_PARAM_RETURN_CODE": "0" }
```

The following values are possible:

Error	Description
"-3"	One of the parameters that were passed contains an invalid value.
"-4"	The signing pad is already being used by another application.
"-5"	No connection has been opened to this signature pad.
"-6"	A connection has already been opened.
"-7"	No further connections can be opened.
"-8"	No device with this ID is connected.
"-12"	The function could not be executed because the signature capture process is running.
"-13"	No further hotspots can be added
"-14"	The coordinates are overlapping with the signature window or one of the hotspots already set.
"-15"	The function could not be executed because no signature capture area has been set.
"-17"	The function could not be executed because no signature capture process was started.
"-18"	An error occurred while attempting to reserve memory.

"-19"	An error occurred while initialising a system resource.
"-20"	An error occurred while communicating with the signing pad.
"-21"	The rectangle that was passed is invalid.
"-22"	No compatible devices connected or the connection to a device has been cut.
"-25"	The connected device does not support this function or one of the parameters.
"-26"	Error while reading or writing a file.
"-84"	The function could not be executed because the secure sign mode is active.
"-97"	An error occurred during initialisation. Please restart the software.
"-400"	Not all mandatory fields have been selected.

8.3.2 TOKEN_PARAM_ERROR_DESCRIPTION key

Available from server version 1.0.0.0 onwards.

The "TOKEN_PARAM_ERROR_DESCRIPTION" key contains an error description as a value.

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":  
  "TOKEN_CMD_OPEN_PAD", "TOKEN_PARAM_PAD_INDEX": "1",  
  "TOKEN_PARAM_RETURN_CODE": "-4", "TOKEN_PARAM_ERROR_DESCRIPTION": "The  
  signing pad is already being used by another application." }
```

8.3.3 TOKEN_PARAM_ERROR_CONTEXT key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_ERROR_CONTEXT" key contains the command in whose context the error occurred.

For an example, see the "TOKEN_CMD_ERROR" value.

8.3.4 TOKEN_PARAM_PAD_SUBSET key

Available from server version 1.0.0.0 onwards.

The "TOKEN_PARAM_PAD_SUBSET" key defines the interfaces to be searched when looking for devices. The value is a list separated by semi-colons with at least one of the following members:

Values	Description
"HID"	A search will be made for HID and WinUSB devices.
"IP=<Address>: <Port>"	An Alpha with an Ethernet connection or a HID device that is connected to a signotec AK USB device server will be searched for at the IP address indicated; <Address> is the IP address of the device server and has the format x.x.x.x or the host name, <Port> is the port via which the signature device should be activated, generally 1002. A valid value would therefore be, for example, "IP=192.168.100.100:1002" or "IP=host1:1002"; for details about the signotec AK USB device server, please refer to your contact at signotec.

"all"	A search will be made for serial devices on all COM ports; this search may take an extremely long time depending on the hardware configuration.
Integer	A search will be made for a serial device on the COM port with the specified number (≥ 0); this search may take an extremely long time if no signotec device is connected to the port
"LowSpeed"	A search will be made on the COM ports only for devices that communicate at a baud rate of 115200 (Sigma, Omega, Gamma plus Delta and Alpha in 'Low Speed' mode). The search is accelerated if COM ports to which no signotec LCD signature pads are connected are also searched; this does not influence the search for HID, WinUSB or IP devices.
"HighSpeed"	A search will be made on the COM ports only for devices that communicate at a baud rate of 2 MBaud (Delta and Alpha in 'High Speed' mode), which accelerates the search; this does not influence the search for HID, WinUSB or IP devices.

For an example, see the "TOKEN_CMD_SEARCH_FOR_PADS" value.

8.3.5 TOKEN_PARAM_CONNECTED_PADS key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_CONNECTED_PADS" key is an array that holds the properties of the connected devices. The array may contain the following keys:

- "TOKEN_PARAM_PAD_INDEX"
- "TOKEN_PARAM_PAD_TYPE"
- "TOKEN_PARAM_PAD_CONNECTION_TYPE"
- "TOKEN_PARAM_PAD_COM_PORT"
- "TOKEN_PARAM_PAD_IP_ADDRESS"
- "TOKEN_PARAM_PAD_SERIAL_NUMBER"
- "TOKEN_PARAM_PAD_FIRMWARE_VERSION"
- "TOKEN_PARAM_PAD_CAPABILITIES" (available from server version 1.3.0.0 onwards)

Example:

```
{
  "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE",
  "TOKEN_CMD_ORIGIN": "TOKEN_CMD_SEARCH_FOR_PADS",
  "TOKEN_PARAM_RETURN_CODE": "16320",
  "TOKEN_PARAM_CONNECTED_PADS": [
    {
      "TOKEN_PARAM_PAD_INDEX": "0",
      "TOKEN_PARAM_PAD_IP_ADDRESS": "",
      "TOKEN_PARAM_PAD_CONNECTION_TYPE": "2",
      "TOKEN_PARAM_PAD_SERIAL_NUMBER": "5",
      "TOKEN_PARAM_PAD_TYPE": "2",
      "TOKEN_PARAM_PAD_FIRMWARE_VERSION": "1.16",
      "TOKEN_PARAM_PAD_COM_PORT": "4",
      "TOKEN_PARAM_PAD_CAPABILITIES": "7874"
    },
    {
      "TOKEN_PARAM_PAD_INDEX": "1",
      "TOKEN_PARAM_PAD_IP_ADDRESS": "",
      "TOKEN_PARAM_PAD_CONNECTION_TYPE": "0",
      "TOKEN_PARAM_PAD_SERIAL_NUMBER": "1000000004",
      "TOKEN_PARAM_PAD_TYPE": "1",
      "TOKEN_PARAM_PAD_FIRMWARE_VERSION": "1.22",
      "TOKEN_PARAM_PAD_COM_PORT": "0",
      "TOKEN_PARAM_PAD_CAPABILITIES": "16320"
    }
  ]
}
```

8.3.6 TOKEN_PARAM_PAD_INDEX key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_INDEX" key is an integer that represents the unique ID of a connected device. The ID remains valid until a new request of type "TOKEN_CMD_SEARCH_FOR_PADS" is sent.

For an example, see the "TOKEN_CMD_OPEN_PAD" value.

8.3.7 TOKEN_PARAM_PAD_TYPE key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_TYPE" key holds the model number of a connected device.

Values	Description
"1"	'Sigma USB' model type
"2"	'Sigma serial' model type
"11"	'Omega USB' model type
"12"	'Omega serial' model type
"15"	'Gamma USB' model type
"16"	'Gamma serial' model type
"21"	'Delta USB' model type
"22"	'Delta serial' model type
"23"	'Delta IP' model type
"31"	'Alpha USB' model type
"32"	'Alpha serial' model type
"33"	'Alpha IP' model type
other	Reserved for further model types

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

8.3.8 TOKEN_PARAM_PAD_CONNECTION_TYPE key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_CONNECTION_TYPE" key contains the type of connection via which a device is connected.

Values	Description
"0"	HID
"1"	WinUSB
"2"	Serial
"3"	Ethernet

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

8.3.9 TOKEN_PARAM_PAD_COM_PORT key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_COM_PORT" key contains the number of the COM port to which a device is connected.

The "TOKEN_PARAM_PAD_COM_PORT" key is not implemented under Linux and has the value 0.

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

8.3.10 TOKEN_PARAM_PAD_IP_ADDRESS key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_IP_ADDRESS" key holds the IP address of a connected device.

The "TOKEN_PARAM_PAD_IP_ADDRESS" key is not implemented under Linux and has the value "".

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

8.3.11 TOKEN_PARAM_PAD_SERIAL_NUMBER key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_SERIAL_NUMBER" key holds the serial number of a connected device.

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

8.3.12 TOKEN_PARAM_PAD_FIRMWARE_VERSION key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_FIRMWARE_VERSION" key holds the firmware version (major.minor) of a connected device.

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

8.3.13 TOKEN_PARAM_FIELD_NAME key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_FIELD_NAME" key can contain any character string that is output at the top left of the signature device's display. This should usually be the name of the signature field that is to be signed.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

8.3.14 TOKEN_PARAM_CUSTOM_TEXT key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_CUSTOM_TEXT" key can contain any character string that is output at the bottom left of the signature device's display.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

8.3.15 TOKEN_PARAM_SIGNATURE_SIGN_DATA key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_SIGNATURE_SIGN_DATA" key holds the SignData of a captured signature. SignData is an encrypted, compressed, biometric data format that can be stored in a database or as a tag in a TIFF or PDF document.

If the command "TOKEN_CMD_SIGNATURE_START" is started with the parameter "TOKEN_PARAM_PAD_ENCRYPTION" = "TRUE" ('RSA in pad' mode), the SignData is encrypted on the pad and the response contains the parameter "TOKEN_PARAM_CERT_ID".

Examples:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":  
"TOKEN_CMD_SIGNATURE_SIGN_DATA", "TOKEN_PARAM_RETURN_CODE": "0",  
"TOKEN_PARAM_SIGNATURE_SIGN_DATA":  
"CgAAAAEAAAABvAAgCVQVVBVUVABAEAAEAQAABA2IYa1x2GVCVU4WzHpbdUCrrInm  
GTLTbzfoOd/WFQo47vAi9Af4AFc=" }
```

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":  
"TOKEN_CMD_SIGNATURE_SIGN_DATA", "TOKEN_PARAM_RETURN_CODE": "0",  
"TOKEN_PARAM_SIGNATURE_SIGN_DATA": "CwAAAAEAAAABmABACwgjCCAAGABu=",  
"TOKEN_PARAM_CERT_ID": "O=Demo signotec, SNR=01" }
```

8.3.16 TOKEN_PARAM_FILE_TYPE key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_FILE_TYPE" key specifies a file type.

Values	Description
"0"	Use TIFF with CCITT4 compression (b/w image) or LZW compression (colour image) as the file format (recommended)
"1"	Use PNG file format
"2"	Use BMP file format
"3"	Use JPEG with a quality setting of 75 as the file format
"4"	Use GIF as the file format

For an example, see the "TOKEN_CMD_SIGNATURE_IMAGE" value.

8.3.17 TOKEN_PARAM_PEN_WIDTH key

Available from server version 1.1.0.0 onwards.

The value of the "TOKEN_PARAM_PEN_WIDTH" key specifies a line width when generating signature images.

Values	Description
< "0"	Fixed stroke width in pixels (absolute value); the pressure values are visualised by drawing in variable brightness
"0"	A variable pen width is used that is dependent on the resolution and the pressure values

> "2"	Fixed pen width in pixels
-------	---------------------------

For an example, see the "TOKEN_CMD_SIGNATURE_IMAGE" value.

8.3.18 TOKEN_PARAM_FILE key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_FILE" key holds a file as a Base64-encoded string.

Example:

```
{ "TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":  
  "TOKEN_CMD_SIGNATURE_IMAGE", "TOKEN_PARAM_RETURN_CODE": "0",  
  "TOKEN_PARAM_FILE":  
    "CgAAAAEAAABvAAgCVQVVBVUVABAEAAEAQAABA2IYa1x2GVCVU4WzHpbdUCrrInm  
    GTLTbzfoOd/WFQo47vAi9Af4AFc=" }
```

8.3.19 TOKEN_PARAM_POINT key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_POINT" key holds an object with information about a captured signature point. The resolution of the data is not suitable for use as biometric data. Please use the SignData instead. You can retrieve this using the "TOKEN_CMD_SIGNATURE_SIGN_DATA" value.

The object may contain the following keys:

Key	Description
"x"	x coordinate between "0" and the display width or vertical signature resolution of the pad model used (see also "TOKEN_PARAM_PAD_DISPLAY_WIDTH" and "TOKEN_PARAM_PAD_X_RESOLUTION" keys).
"y"	y coordinate between "0" and the display height or horizontal signature resolution of the pad model used (see also "TOKEN_PARAM_PAD_DISPLAY_HEIGHT" and "TOKEN_PARAM_PAD_Y_RESOLUTION" keys).
"p"	"0" defines the point of setting down the pen (first point of a signature line). "1" defines a pen that is set down (all further points of a signature line).

For an example, see the "TOKEN_CMD_SIGNATURE_POINT" key.

8.3.20 TOKEN_PARAM_VERSION key

Available from server version 1.0.2.4 onwards.

The value of the "TOKEN_PARAM_VERSION" key must be in the format "major.minor.revision.build".

Permitted values are:

Part	Permitted values
"major"	0 - 65535
"minor"	0 - 65535
"revision"	0 - 65535
"build"	0 - 65535

For the "TOKEN_CMD_SET_INTERFACE_VERSION" command, the parameter sets the version for the supported server interface. For the "TOKEN_CMD_GET_SERVER_VERSION" command, the parameter returns the server version.

For examples, see the "TOKEN_CMD_SET_INTERFACE_VERSION" and "TOKEN_CMD_GET_SERVER_VERSION" values.

8.3.21 TOKEN_PARAM_PAD_X_RESOLUTION key

Available from server version 1.0.2.4 onwards.

The value of the "TOKEN_PARAM_PAD_X_RESOLUTION" key is sent with the answer to the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" commands. The value of the "TOKEN_PARAM_PAD_X_RESOLUTION" key is the X resolution of the virtual screen for the real-time signature display. The X coordinate in the "TOKEN_CMD_SIGNATURE_POINT" event relates to the value of the "TOKEN_PARAM_PAD_X_RESOLUTION" key from server version 1.0.2.4 onwards.

For examples, see the "TOKEN_CMD_OPEN_PAD", "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" and "TOKEN_CMD_SIGNATURE_POINT" values.

8.3.22 TOKEN_PARAM_PAD_Y_RESOLUTION key

Available from server version 1.0.2.4 onwards.

The value of the "TOKEN_PARAM_PAD_Y_RESOLUTION" key is sent with the answer to the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" commands. The value of the "TOKEN_PARAM_PAD_Y_RESOLUTION" key is the Y resolution of the virtual screen for the real-time signature display. The Y coordinate in the "TOKEN_CMD_SIGNATURE_POINT" event relates to the value of the "TOKEN_PARAM_PAD_Y_RESOLUTION" key from server version 1.0.2.4.

For examples, see the "TOKEN_CMD_OPEN_PAD", "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" and "TOKEN_CMD_SIGNATURE_POINT" values.

8.3.23 TOKEN_PARAM_PAD_DISPLAY_WIDTH key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_DISPLAY_WIDTH" key is sent with the answer to the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_API_DISPLAY_GET_WIDTH" commands. The value of the "TOKEN_PARAM_PAD_DISPLAY_WIDTH" key is the width of the screen of a signotec LCD signature pad. The X coordinate in the "TOKEN_CMD_SIGNATURE_POINT" event relates to the value of the "TOKEN_PARAM_PAD_DISPLAY_WIDTH" key up to server version 1.0.2.3.

For examples, see the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_SIGNATURE_POINT" values.

8.3.24 TOKEN_PARAM_PAD_DISPLAY_HEIGHT key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_DISPLAY_HEIGHT" key is sent with the answer to the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_API_DISPLAY_GET_HEIGHT" commands. The value of the "TOKEN_PARAM_PAD_DISPLAY_HEIGHT" key is the width of the screen of a signotec LCD signature pad. The Y coordinate in the "TOKEN_CMD_SIGNATURE_POINT" event relates to the value of the "TOKEN_PARAM_PAD_DISPLAY_HEIGHT" key up to server version 1.0.2.3.

For examples, see the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_SIGNATURE_POINT" values.

8.3.25 TOKEN_PARAM_PAD_SAMPLING_RATE key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_SAMPLING_RATE" key is sent with the answer to the "TOKEN_CMD_OPEN_PAD" command. The value of the "TOKEN_PARAM_PAD_SAMPLING_RATE" key is the sample rate of the sensor of a signotec LCD signature pad.

Permitted values
125 Hz
250 Hz
500 Hz
280 Hz

For an example, see the "TOKEN_CMD_OPEN_PAD" value.

8.3.26 TOKEN_PARAM_PAD_CAPABILITIES key

Available from server version 1.3.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_CAPABILITIES" key is sent with the answer to the "TOKEN_CMD_SEARCH_FOR_PADS" command. The value of the "TOKEN_PARAM_PAD_CAPABILITIES" key is a whole number whose value corresponds to the return value of the STDeviceGetCapabilities() method.

For details, please see the documentation of the STDeviceGetCapabilities() method in the signoPAD API.

For examples, see the "TOKEN_PARAM_CONNECTED_PADS" values.

8.3.27 TOKEN_PARAM_FIELD_LIST key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_LIST" defines the selection fields and their texts for the selection dialog that can be configured with the command "TOKEN_CMD_SELECTION_DIALOG". Its value must be an array that can contain the following keys:

- "TOKEN_PARAM_FIELD_ID"
- "TOKEN_PARAM_FIELD_TEXT"
- "TOKEN_PARAM_FIELD_CHECKED"
- "TOKEN_PARAM_FIELD_REQUIRED"

For examples, see the "TOKEN_CMD_SELECTION_DIALOG" value.

8.3.28 TOKEN_PARAM_FIELD_ID key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_ID" defines the ID of a selection field or contains the previously defined ID. The ID can be any value, but must not already have been used on another field.

For examples, see the "TOKEN_CMD_SELECTION_DIALOG" and "TOKEN_CMD_SELECTION_CHANGE" values.

8.3.29 TOKEN_PARAM_FIELD_TEXT key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_TEXT" defines the text of a selection field. It can have any length and may contain "\n" for a new paragraph.

For examples, see the "TOKEN_CMD_SELECTION_DIALOG" value.

8.3.30 TOKEN_PARAM_FIELD_CHECKED key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_CHECKED" defines if a selection field is checked.

Permitted values:

Value	Meaning
"TRUE"	Selection field is checked.
"FALSE"	Selection field is not checked.

For examples, see the "TOKEN_CMD_SELECTION_DIALOG" and "TOKEN_CMD_SELECTION_CHANGE" values.

8.3.31 TOKEN_PARAM_FIELD_REQUIRED key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_REQUIRED" defines whether a selection field must be selected in order to confirm the selection dialog. If the 'Confirm' button is pressed without all required fields being selected, an event is triggered with the command "TOKEN_CMD_ERROR" and the "TOKEN_PARAM_RETURN_CODE" "-400".

Permitted values:

Value	Meaning
"TRUE"	Selection field must be checked (required field).
"FALSE"	Selection field doesn't have to be checked (optional field).

For examples, see the "TOKEN_CMD_SELECTION_DIALOG" value.

8.3.32 TOKEN_PARAM_PAD_ENCRYPTION key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_PAD_ENCRYPTION" key sets 'RSA in pad' mode. If it contains "TRUE", the WebSocket Pad Server will be switched to 'RSA in pad' mode. 'RSA in pad' mode remains disabled in all other cases. Before the server is switched to 'RSA in pad' mode, the device properties are checked to determine if the device supports encryption in the pad. A check is then performed (based on whether the parameter "TOKEN_PARAM_ENCRYPTION_CERT" was transferred) to determine if the pad has stored a biometric key or an attempt is made to upload the key transferred with "TOKEN_PARAM_ENCRYPTION_CERT" to the pad. An error is generated if it is determined that encryption is not possible in the pad.

For examples, see the "TOKEN_CMD_SIGNATURE_START" values.

8.3.33 TOKEN_PARAM_DOCHASH key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_DOCHASH" key is only evaluated if the "TOKEN_PARAM_PAD_ENCRYPTION" key contains the value "TRUE". The "TOKEN_PARAM_DOCHASH" key contains a Base64 string that is a 20 (SHA1) or 32 (SHA256) byte array when decoded. An error is generated with other content. The byte array includes a checksum (typically the hash of a document to be signed). The encoded byte array is transferred to the pad.

For examples, see the "TOKEN_CMD_SIGNATURE_START" values.

8.3.34 TOKEN_PARAM_ENCRYPTION_CERT key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_ENCRYPTION_CERT" key is only evaluated if the "TOKEN_PARAM_PAD_ENCRYPTION" key contains the value "TRUE". The "TOKEN_PARAM_ENCRYPTION_CERT" key contains a Base64 string that is an X.509 certificate when decoded. The certificate contains the public key used to encrypt the biometric data on the pad. The encoded certificate the key contains is uploaded to the pad.

For examples, see the "TOKEN_CMD_SIGNATURE_START" values.

8.3.35 TOKEN_PARAM_CERT_ID key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_CERT_ID" key is only returned if the "TOKEN_PARAM_PAD_ENCRYPTION" key contains the value "TRUE". The returned value is the ID of the public key that is stored in the signature device and used for the encryption of biometric data. The ID is a character string consisting of the 'Issuer' and 'Serial Number' of the certificate in hexadecimal representation that has been transferred when saving the key, e.g., 'Example certificate (ff 0a)'

For examples, see the "TOKEN_PARAM_SIGNATURE_SIGN_DATA" values.

8.3.36 TOKEN_PARAM_ROTATION key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_ROTATION" key defines the rotation with which the image data is transferred to the signature device and the sensor areas are defined. A change to the value only affects images and sensor areas which are transferred after the change is made and not images which are copied from one pad memory to another. The default setting is 0.

For examples, see the "TOKEN_CMD_API_DISPLAY_SET_ROTATION" and "TOKEN_CMD_API_DISPLAY_GET_ROTATION" values.

8.3.37 Other keys

All other keys can be used as request parameters in API mode. For a description, please see the corresponding method in the signoPAD API documentation.